

Objects That Remember

CONTENTS

01	Comprehensions	list, set, dict, generator
02	Why a class rather than a function	state between calls
03	Writing a class	<code>__init__</code> , <code>self</code> , attributes
04	What the built-ins actually call	<code>__len__</code> , <code>__contains__</code> , <code>__getitem__</code>
05	The iterator protocol	<code>__iter__</code> , <code>__next__</code> , <code>StopIteration</code>
06	Generators	<code>yield</code> , <code>yield from</code> , laziness
07	Choosing between them	list, generator, class

What you should be able to do

A function forgets everything the moment it returns, and a list holds everything at once. Between those two limits sit the two constructions below: an object that keeps something between calls, and a function that hands back values one at a time instead of all at once. Both are ordinary Python, and both are what libraries are built out of — almost every object you use from a package is one, and almost every dataset too large for memory is walked lazily.

Without looking anything up:

1. Write a class with `__init__` and methods that change its own state.
2. Say what `len(x)`, `x in y` and `y[i]` actually call, and implement `__len__`, `__contains__` and `__getitem__` so those built-ins work on an object you wrote.
3. Implement the iterator protocol by hand: `__iter__`, `__next__`, and raising `StopIteration`.
4. Write a generator with `yield`, and explain why it does not build a list.
5. Recognise when laziness matters, and when it costs you.

Every item on that list is something already used constantly from the outside: a `for` loop over a file, `len` on a list, a `range` that does not fill memory. What follows is the other side of those — the methods Python calls when you write them.

01

Comprehensions

A comprehension is an expression that builds a container by looping. It is shorter than the loop it replaces, and it costs exactly the same.

THE FOUR FORMS

```
nums = [4, -1, 7, -9, 0, -3, 5]

squares = [x * x for x in nums]           # list comprehension -> list
distinct = {x * x for x in nums}          # set comprehension -> set
sized    = {w: len(w) for w in words}     # dict comprehension -> dict, key: value
lazy     = (x * x for x in nums)          # generator expression -> generator
```

The first three build the whole container immediately. The fourth builds nothing yet, and is the subject of section 6.

A comprehension is always equivalent to a loop, and it helps to be able to write the loop back out. These two are the same program:

```
kept = [x for x in nums if x >= 0]
```

```
kept = []
for x in nums:
    if x >= 0:
        kept.append(x)
```

The optional `if` at the end filters, so the result can be shorter than the input. It is also the efficient way to remove elements from a list: building a new list is one pass, while deleting from the middle of an existing one shifts every later element down and costs a pass of its own each time.

WHEN TO WRITE THE LOOP INSTEAD

A comprehension holds exactly one expression. The moment the body needs a second statement, a temporary variable, or a `try`, the loop is the correct form and the comprehension stops being shorter. Two more cases where the loop wins:

- **Nesting past two levels.** A comprehension with two `for` clauses is readable. Three is not, and the reading order is a common surprise: the clauses run left to right, exactly as the nested loop would.
- **Side effects.** A comprehension is for producing a value.

```
[print(x) for x in nums]    # builds a list of None and throws it away
```

```
for x in nums:              # what was meant
    print(x)
```

THE READING ORDER OF A NESTED COMPREHENSION

The `for` clauses appear in the same order as the nested loops they stand for, but the output expression sits at the front, which is what makes it look reversed.

```
flat = [x for row in grid for x in row]

flat = []
for row in grid:          # first clause is the outer loop
    for x in row:          # second clause is the inner loop
        flat.append(x)
```

02

Why a class rather than a function

A function forgets everything when it returns. When the answer to a question depends on what happened earlier, that information has to live somewhere.

Consider a counter. As a function it does not work, because the count has nowhere to go:

```
def count():
    total = 0          # created fresh on every call
    total += 1
    return total

count(), count(), count()    # (1, 1, 1)
```

The usual first repair is a global variable, and it is worse than it looks. There is only one of it, so two independent counters are impossible, and any part of the program can change it without the function knowing.

```
total = 0
```

```
def count():
    global total
    total += 1
    return total
```

A class solves both problems at once. The data lives inside an object, each object has its own, and the methods that are allowed to change it sit next to it:

```
class Counter:
    def __init__(self, start=0):
        self.total = start

    def count(self):
        self.total += 1
        return self.total

a = Counter()
b = Counter(100)
a.count(), a.count(), b.count()      # (1, 2, 101)
```

`a` and `b` are separate objects with separate state. That is the whole reason to reach for a class: **you need more than one of something, and each one has to remember its own history.**

WHEN A FUNCTION IS STILL THE RIGHT ANSWER

If the output depends only on the arguments, write a function. A class whose `__init__` stores nothing and whose methods never read `self` is a module with extra ceremony. The question to ask is whether two calls with the same arguments must be able to give different answers. If not, no state is needed.

03

Writing a class

A class is a template. An instance is one object built from it. Everything the instance remembers lives in attributes reached through `self`.

__INIT__ AND SELF

`__init__` runs immediately after a new instance is created, and its job is to set up that instance's starting data. It is not quite a constructor: the object already exists by the time it runs, and it returns nothing.

`self` is the instance the method was called on. Python passes it automatically as the first argument, which is why it appears in the definition but not at the call site:

```
c = Counter(10)      # Counter.__init__(c, 10) runs behind this
c.count()            # Counter.count(c)
```

Attributes are created by assigning to them. There is no separate declaration, and nothing stops you adding one later, though doing it outside `__init__` makes an object whose shape depends on which methods happened to run.

```
class Playlist:
    def __init__(self, name):
        self.name = name          # these two lines create the attributes
        self.tracks = []

    def add(self, title):
        self.tracks.append(title)
        return self               # returning self allows p.add("a").add("b")
```

```
def total(self):
    return len(self.tracks)
```

The name `self` is a convention, not a keyword. Any name works in that position, and every reader will be confused, so it is never done.

INSTANCE ATTRIBUTES AGAINST CLASS ATTRIBUTES

A name assigned inside `__init__` through `self` belongs to that one instance. A name assigned directly in the class body belongs to the class and is shared by every instance of it.

```
class Track:
    medium = "digital"           # class attribute: one copy, shared

    def __init__(self, title):
        self.title = title       # instance attribute: one copy per track
```

Sharing is harmless for a constant. It is a defect when the shared object can be modified, because every instance then sees the change. This is the single most common bug in a first class, and it produces no error at all:

```
class Playlist:
    tracks = []                  # ONE list, shared by every playlist ever made

    def add(self, title):
        self.tracks.append(title)

a, b = Playlist(), Playlist()
a.add("Kind of Blue")
b.tracks                          # ['Kind of Blue'], although nothing was added to b
```

The fix is to create the list per instance, inside `__init__` :

```
class Playlist:
    def __init__(self):
        self.tracks = []        # a new list for each instance
```

THE SAME TRAP, IN A FUNCTION

A default argument is evaluated once, when the `def` is executed, not once per call. A mutable default is therefore shared between calls in exactly the way a mutable class attribute is shared between instances.

```
def collect(item, into=[]):      # this list is created once, ever
    into.append(item)
    return into

collect(1)                       # [1]
collect(2)                       # [1, 2], not [2]
```

```
def collect(item, into=None):    # the standard repair
    if into is None:
        into = []
    into.append(item)
    return into
```

MAKING AN OBJECT PRINTABLE

By default, printing an object gives you its type and memory address, which tells you nothing while debugging. `__repr__` replaces that with something you choose. Write it early; it pays for itself the first time a test fails.

```
class Playlist:
    def __init__(self, name):
        self.name = name
        self.tracks = []

    def __repr__(self):
        return f"Playlist({self.name!r}, {len(self.tracks)} tracks)"

Playlist("Roadtrip")           # Playlist('Roadtrip', 0 tracks)
```

Two details in that line. The `f` prefix marks an **f-string**, in which an expression inside braces is evaluated and inserted. The `!r` asks for the `repr` of a value rather than its `str`, which is what puts the quotation marks around the name and makes the output look like code.

04

What the built-ins actually call

Methods named with double underscores at both ends are called **dunder methods**. You do not call them yourself. Python calls them when the matching syntax or built-in function is used, so implementing one makes your object respond to ordinary language constructs.

YOU WRITE	PYTHON CALLS	PURPOSE
<code>len(x)</code>	<code>x.__len__()</code>	The number of items
<code>v in x</code>	<code>x.__contains__(v)</code>	Membership
<code>x[i]</code>	<code>x.__getitem__(i)</code>	Indexing
<code>x[i] = v</code>	<code>x.__setitem__(i, v)</code>	Index assignment
<code>x == y</code>	<code>x.__eq__(y)</code>	Equality
<code>repr(x)</code>	<code>x.__repr__()</code>	The text shown in a console
<code>str(x)</code> , <code>print(x)</code>	<code>x.__str__()</code>	Text for a human, falls back to <code>__repr__</code>
<code>for v in x</code>	<code>x.__iter__()</code>	Iteration, section 5

Three of them turn the playlist into something that behaves like a built-in container:

```
class Playlist:
    def __init__(self, name):
        self.name = name
        self.tracks = []

    def add(self, title):
        self.tracks.append(title)

    def __len__(self):
        return len(self.tracks)

    def __contains__(self, title):
        return title in self.tracks

    def __getitem__(self, i):
        return self.tracks[i]
```

```

p = Playlist("Roadtrip")
p.add("Kind of Blue")
p.add("So What")

len(p)           # 2,      through __len__
"So What" in p   # True,   through __contains__
p[0]             # 'Kind of Blue', through __getitem__
p[-1]            # 'So What', because list indexing handles it for you

```

Note what the last two lines cost you: nothing. `__getitem__` forwards to the list, so negative indices and slices keep working without another line of code.

A PARTIAL PROTOCOL IS FINE

These methods are independent. A class with `__len__` but no `__getitem__` answers `len(x)` and raises `TypeError` on `x[0]`. Python checks for the specific method at the moment the syntax is used, so an object supports exactly the operations it has methods for and nothing has to be declared in advance.

EQUALITY, AND THE RULE THAT COMES WITH IT

By default two objects are equal only if they are the same object. Defining `__eq__` changes that to whatever you say:

```

class Track:
    def __init__(self, title, artist):
        self.title = title
        self.artist = artist

    def __eq__(self, other):
        if not isinstance(other, Track):
            return NotImplemented
        return (self.title, self.artist) == (other.title, other.artist)

```

Returning `NotImplemented` for an unrelated type is the correct move rather than `False`: it tells Python to try the comparison the other way round before giving up.

There is a catch, and it is worth remembering because the failure is silent until it is not. **Defining `__eq__` removes the default `__hash__`**, so instances can no longer go in a set or be used as dictionary keys:

```

{Track("So What", "Miles Davis")}      # TypeError: unhashable type: 'Track'

```

Python does this deliberately. A hash table only works if equal objects hash the same and a key's hash never changes. Having redefined what equal means, you have to say what hashing means too, and it must be built from the same fields:

```

def __hash__(self):
    return hash((self.title, self.artist))

```

If the object is meant to be mutable, leaving it unhashable is the honest outcome, and the same reasoning is why `list` is unhashable and `tuple` is not.

05

The iterator protocol

A `for` loop is not a primitive. It is built on a protocol that any object can implement, and knowing the protocol explains several things that otherwise look like magic.

An **iterable** is anything you can loop over: a list, a string, a dictionary, an open file. An **iterator** is the object that does the walking and remembers the position. Two built-in functions connect them.

iter(x)

Asks the iterable `x` for a fresh iterator, by calling its `__iter__` method.

`next(it)`

Asks the iterator for the next item, by calling its `__next__` method. When nothing remains it raises `StopIteration` rather than returning a value.

So this loop:

```
for item in nums:
    print(item)
```

is shorthand for exactly this:

```
it = iter(nums)
while True:
    try:
        item = next(it)
    except StopIteration:
        break
    print(item)
```

Since that is all a `for` loop requires, any object providing those two methods can be looped over. Written by hand:

```
class CountUpTo:
    def __init__(self, limit):
        self.limit = limit
        self.current = 0

    def __iter__(self):
        # hand back the object that walks
        return self

    def __next__(self):
        # produce one item, or signal the end
        if self.current >= self.limit:
            raise StopIteration
        self.current += 1
        return self.current

for n in CountUpTo(3):
    print(n)                # 1, 2, 3
```

ITERABLE OR ITERATOR: THE DIFFERENCE THAT BITES

An object with both methods is an **iterator**, and its `__iter__` returns `self`. An object with only `__iter__` is an **iterable** that hands out a separate iterator each time. That distinction decides whether the object can be looped over twice.

```
class Countdown:
    # an ITERABLE: a fresh iterator per loop
    def __init__(self, start):
        self.start = start

    def __iter__(self):
        return iter(range(self.start, 0, -1))

c = Countdown(3)
list(c)        # [3, 2, 1]
list(c)        # [3, 2, 1] again, because each loop got its own iterator
```

```
c = CountUpTo(3)           # an ITERATOR: one pass, then finished
list(c)                    # [1, 2, 3]
list(c)                    # [], because the state was consumed by the first pass
```

A list is an iterable and not an iterator: it has no `__next__`, and `iter(mylist)` returns a separate list-iterator object every time it is called. That is exactly why a list can be walked repeatedly and a generator cannot.

A SHORTCUT WORTH KNOWING

If a class defines `__getitem__` that accepts 0, 1, 2 and raises `IndexError` when it runs out, Python will iterate it even with no `__iter__` at all. This is the older sequence protocol, still supported. The playlist from section 4 is already iterable for this reason.

06

Generators

Writing the iterator class in section 5 took eleven lines to count to three. A generator is the short route to the same object.

A function containing `yield` is a **generator function**. Calling it does not run the body: it returns a generator object, which is an iterator. The body runs only as items are requested, pauses at each `yield`, and keeps its local variables alive until the next request.

```
def count_up_to(limit):
    current = 0
    while current < limit:
        current += 1
        yield current           # pause here, resume on the next request

for n in count_up_to(3):
    print(n)                   # 1, 2, 3, exactly the class from section 5
```

Nothing about the loop changed. The generator supplies `__iter__` and `__next__`, and raises `StopIteration` when the function body ends, all without you writing any of it.

WHAT A PAUSED GENERATOR HOLDS

The class version stored its position in `self.current`, by hand. The generator stores it in the paused frame: every local variable, and the exact line it stopped on. That is the whole difference, and it is why the generator is shorter rather than merely tidier.

It also explains the memory behaviour. A generator never holds more than the item it is producing at that moment, so it can stand for a sequence larger than memory, or one with no end at all:

```
def naturals():               # infinite, and perfectly safe
    n = 0
    while True:
        n += 1
        yield n

squares = (n * n for n in naturals())
[next(squares) for _ in range(5)]  # [1, 4, 9, 16, 25]
```

YIELD FROM

`yield from other_iterable` yields every item that iterable produces, one at a time, as if they had been yielded here directly. It is a delegation, not a call that returns a value.

```
def letters():
    yield "a"
    yield "b"
```

```
def both():
    yield from letters()      # delegate: a, then b
    yield from letters()      # and again

list(both())                  # ['a', 'b', 'a', 'b']
```

Without it, the same delegation is a loop, which is why `yield from` exists:

```
def both():
    for item in letters():
        yield item
    for item in letters():
        yield item
```

It matters most for recursion. Walking a structure that nests to an unknown depth is four lines with `yield from` and considerably more without:

```
def flatten(items):
    for item in items:
        if isinstance(item, list):
            yield from flatten(item)
        else:
            yield item

list(flatten([1, [2, [3, 4]], 5]))  # [1, 2, 3, 4, 5]
```

THE MISTAKE THAT RAISES NOTHING

Calling a recursive generator without `yield from` or a loop builds a generator object and throws it away. No exception is raised, the body never runs, and every nested value silently vanishes from the output.

```
def flatten(items):
    for item in items:
        if isinstance(item, list):
            flatten(item)      # builds a generator, discards it
        else:
            yield item

list(flatten([1, [2, [3, 4]], 5]))  # [1, 5] and no error
```

WHAT LAZINESS COSTS

A generator is single pass. Once read, the sequence cannot be walked again, its length is unknown without consuming it, and there is no random access. A list supports all three, because it paid for real memory in advance.

```
g = (x * x for x in [1, 2, 3])
list(g)      # [1, 4, 9]
list(g)      # [], the first pass exhausted it
len(g)       # TypeError: object of type 'generator' has no len()
```

So the choice is not about elegance. It is whether the sequence is read once from front to back, in which case laziness is free, or repeatedly and out of order, in which case building the list is correct and the generator is the wrong tool.

RECURSION DEPTH

Python limits recursion to roughly a thousand frames and raises `RecursionError` beyond it. A recursive generator uses one frame per level of nesting, which is safe for any realistic structure and would fail on one nested a thousand deep. An explicit stack in a list has no such limit, since it lives on the heap.

07

Choosing between them

Three ways to hand back a sequence, and one way to hold state. The decision is usually made by how the result will be read, not by how it will be written.

WRITE	MEMORY	READ TWICE	REACH FOR IT WHEN
List comprehension	$O(n)$	yes	The result is small, or is needed more than once, indexed, or measured with <code>len</code>
Generator expression	$O(1)$	no	It feeds straight into <code>sum</code> , <code>max</code> , <code>any</code> or a single loop
Generator function	$O(1)$	no	Producing items needs several statements, recursion, or has no natural end
Class	what you store	yes	Several operations share state, or you need many independent copies of it

A useful check for the middle two: if you find yourself writing `list(...)` around a generator immediately, the list comprehension was the right thing to write. If you find yourself building a list only to loop over it once and discard it, the generator was.

WHERE THE PROTOCOL SHOWS UP

Training data rarely fits in memory, so it is loaded in batches: something produces one batch at a time, on request, and never holds the whole set. That is a generator. PyTorch's `DataLoader` is this protocol underneath, and so is reading a multi-gigabyte log file a line at a time rather than calling `read()` on it.

Further reading

- The classes tutorial (<https://docs.python.org/3/tutorial/classes.html>), and its section on iterators and generators (<https://docs.python.org/3/tutorial/classes.html#iterators>).
- The data model reference (<https://docs.python.org/3/reference/datamodel.html>): the full list of dunder methods, of which sections 4 and 5 above cover the handful used most.
- `itertools` (<https://docs.python.org/3/library/itertools.html>), a standard-library module of generators for the patterns that recur: chaining sequences, grouping runs, slicing, and taking the first few items of an infinite one.

The short version

A class is for state that several operations share. A generator is for a sequence you read once. The dunder methods are how your own objects join in with syntax that already exists rather than inventing their own. Everything else follows from those three sentences.

